
Contents

Il processore RISC-V	2
Data path	2
Fetch-Decode-Execute	3
Fetch	3
Decode	4
Execute	5
Temporizzazione	7
Unità di controllo	8
Controllo dell'ALU	9
Unità di controllo principale	12
Il bus	16
Dispositivi attivi e passivi	18
Progettazione di un bus	18
Bus clocking	18
Bus Sincrono	20
Bus Asincrono	21
Arbitraggio del bus	21
Bus paralleli e seriali	22

Il processore RISC-V

Data path

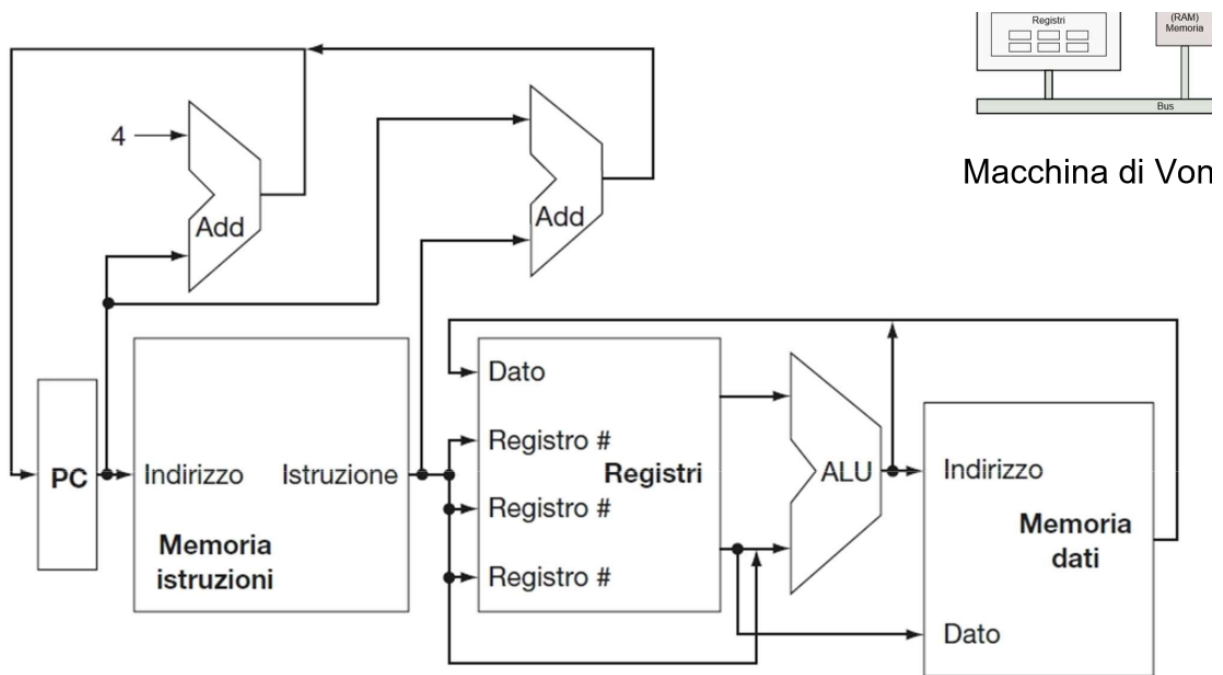


Figure 1: alt text

Memoria istruzioni e dati sono uniche nella memoria centrale di un calcolatore, in questo disegno sono separate per rendere lo schema più comprensibile.

I circuiti combinatori Add, segnati come ALU, fanno solo somme. Il primo add si occupa di aumentare il PC per il ciclo fetch. ($PC+4$). Il secondo Add si occupa di aumentare il PC nel caso dei branch per andare all'etichetta richiesta.

Fetch-Decode-Execute

Fetch

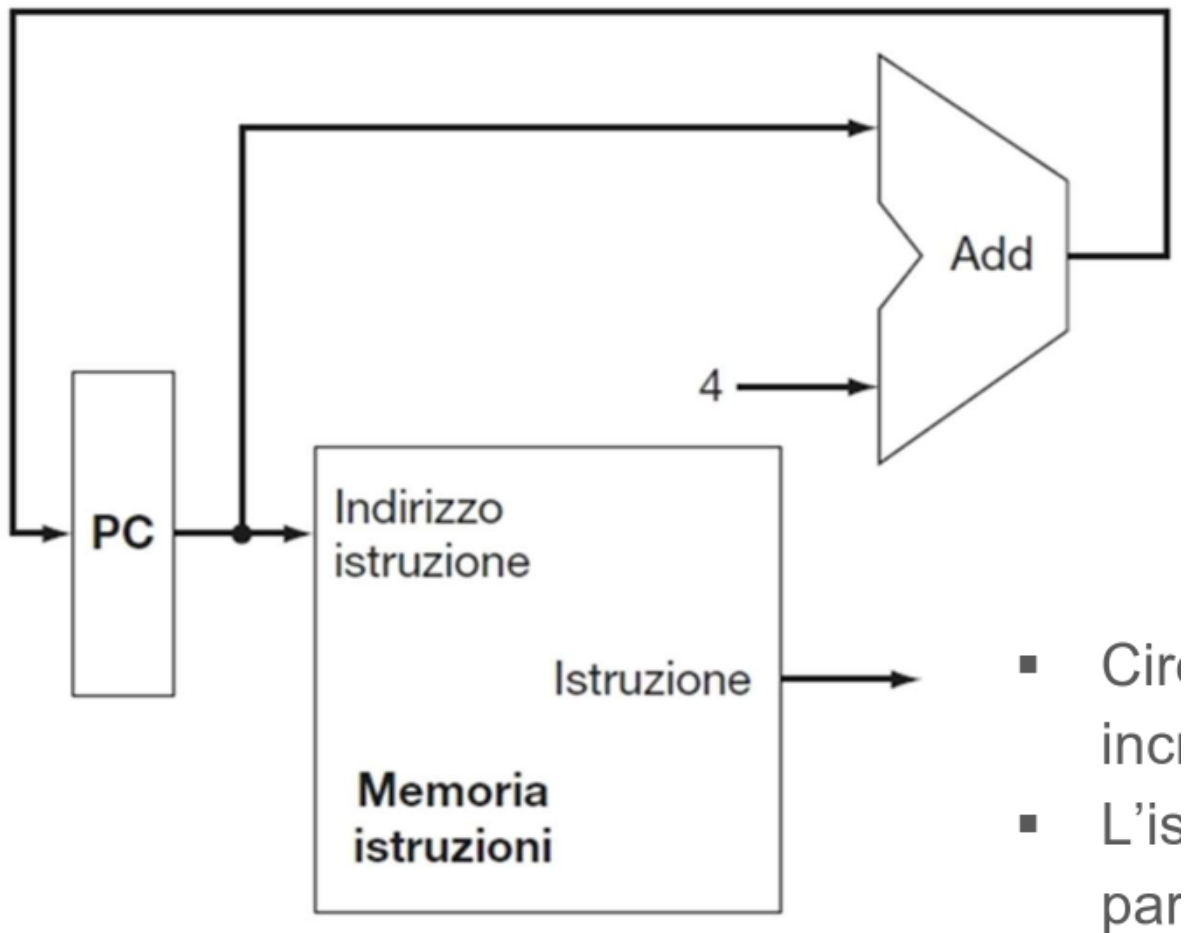


Figure 2: alt text

Questa parte si occupa del fetch, cioè di prendere le istruzioni e passarle alla parte di decode.
è composto da:

- Un registro PC che viene incrementato di 4
- Una memoria per le istruzioni

L'aumento del PC è parallelo al fetch dell'istruzione grazie alla temporizzazione data dal clock. Infatti dopo aver aumentato il PC, il valore $PC+4$ rientra nell'addizionatore e aumenterà nuovamente.

Le istruzioni finisco all'unità di controllo per il decode.

Decode

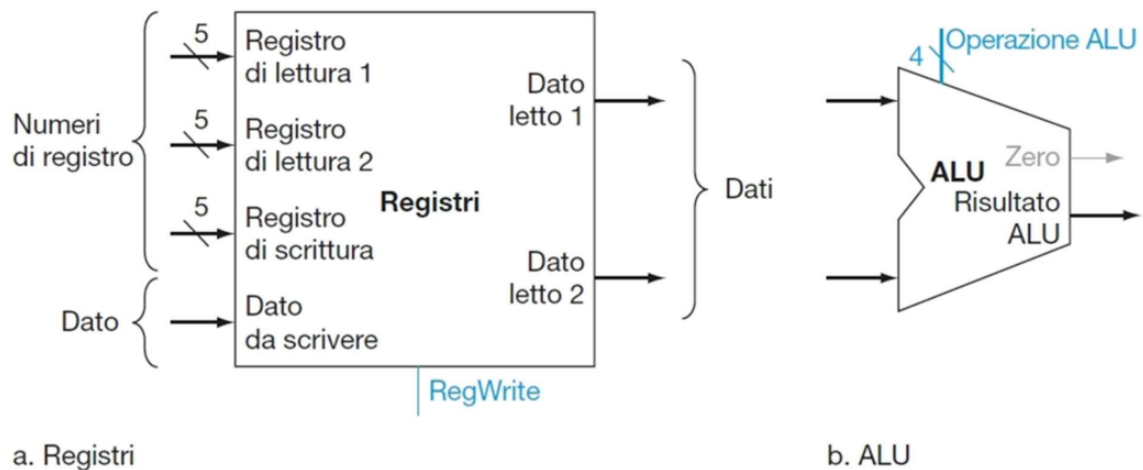


Figure 3: alt text

Le istruzioni prese dalla memoria, passano all'unità di controllo che controlla l'ALU che a sua volta dovrà interpretare l'istruzione per capire l'operazione richiesta. L'unità di controllo (non presente nell'immagine) fa il decode dell'istruzione. Ad esempio l'unità di controllo pilota il register file, l'ALU...

Ad esempio l'unità di controllo data un'istruzione di tipo R, pilota l'ALU dandogli 4 bit per rappresentare l'operazione richiesta.

Sempre in formato R, l'unità di controllo manderà rs1, rs2 e rd al register file per accedere ai registri richiesti.

Inoltre controllerà anche RegWrite in maniera temporizzata quando l'ALU sarà stabile (non è controllata dal clock). Sul fronte attivo del clock, l'ALU dovrà comunque essere stabile.

Execute

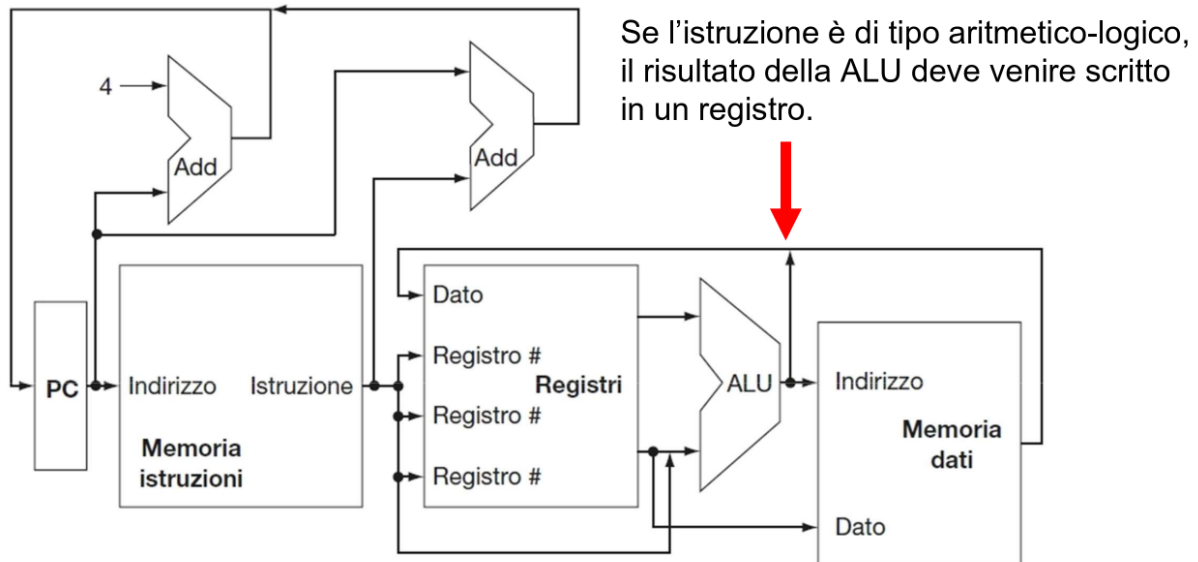


Figure 4: CPU per Formato R

Una volta che l'ALU riceve gli operandi dal register file, esegue l'operazione richiesta.

Nel caso di load o store il valore calcolato dall'ALU non va salvato in un registro ma passa alla memoria dati.

Per le istruzioni I e S ci servono 2 nuovi circuiti digitali:

- La memoria dati
- Il generatore di costanti. Questo circuito prende i 12 bit di costante (che non sono sempre contigui) nell'istruzione I o S e la converte in 32 bit con estensione del segno. Anche nel formato SB verrà usato questo circuito.

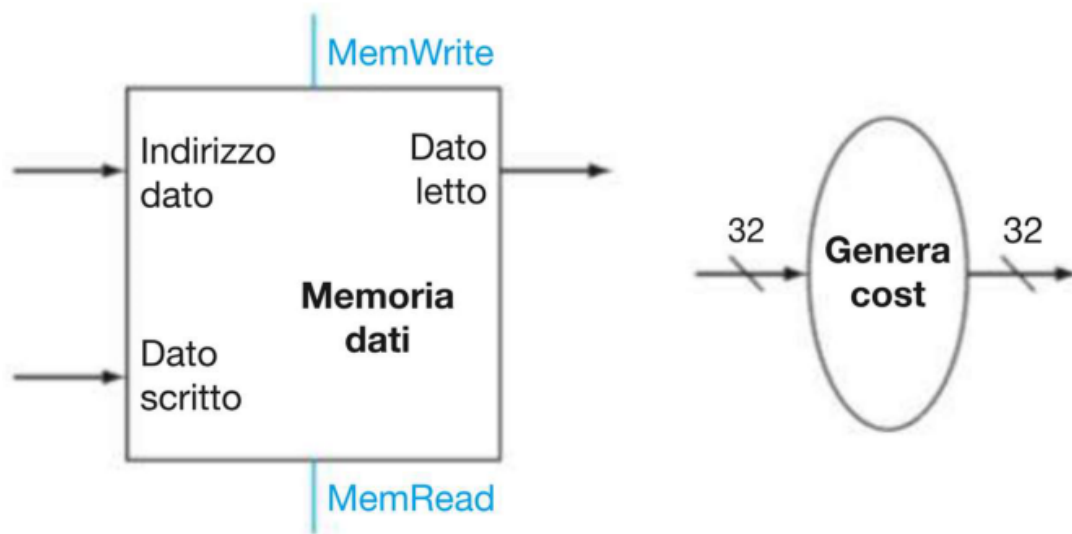


Figure 5: alt text

Il tutto è controllato dall'unità di controllo.

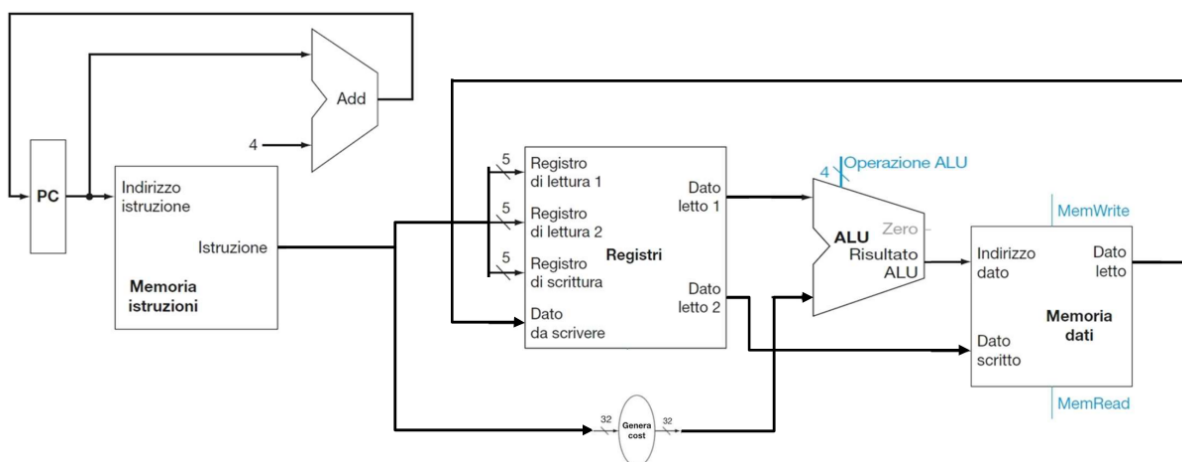


Figure 6: CPU per formato S e I

Istruzioni SB Per le istruzioni SB dobbiamo aggiungere un sommatore per il PC.

Per unire tutto insieme usiamo i multiplexer:

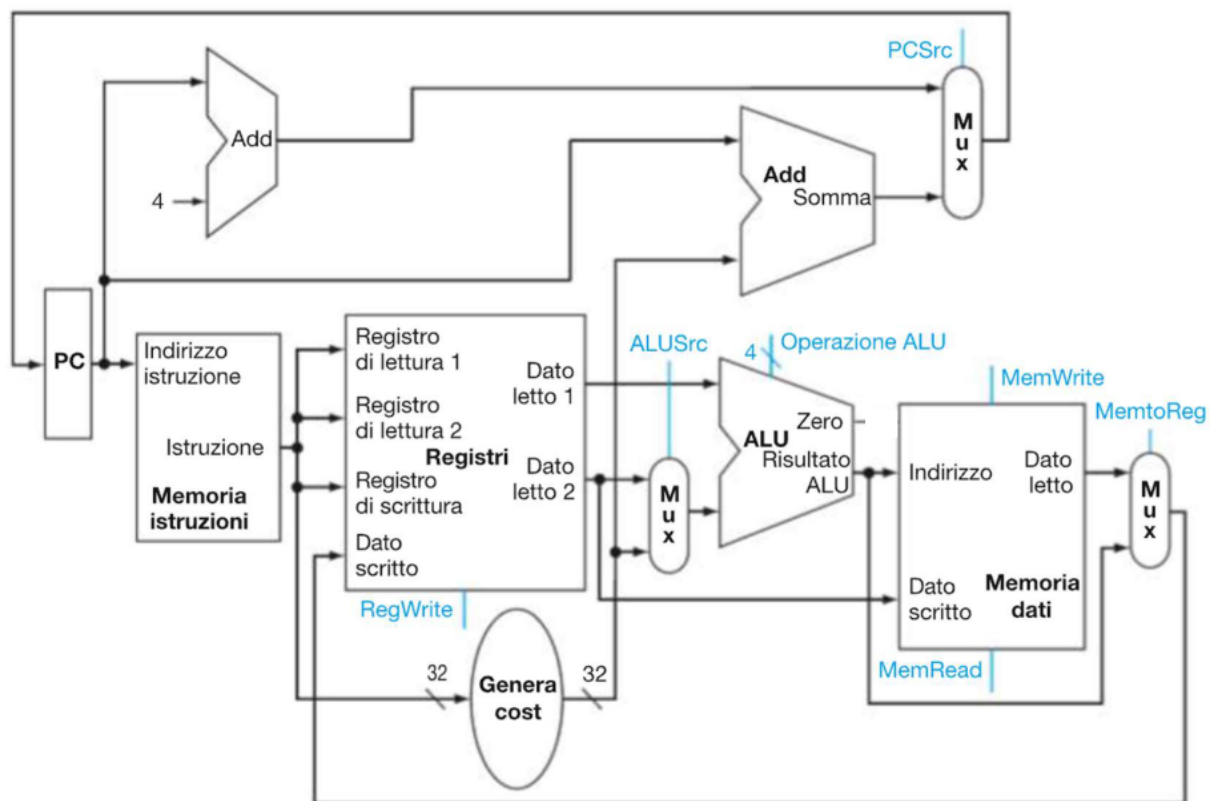


Figure 7: alt text

Tutti i multiplexer saranno gestiti dall'unità di controllo sulla base dell'unità di controllo.

Una CPU di questo genere riesce a compiere semplici operazioni come add, sub, beq, load, store

Temporizzazione

Tutti i circuiti che hanno bisogno di temporizzazione(registri, memoria) usano la stessa linea di clock.

Di solito si usa la temporizzazione edge-triggered, i cambiamenti di stato avvengono ai fronti(temporizzazione sensibile ai fronti).

I circuiti devono essere stabili prima del fronte del clock.

Usando questa temporizzazione possiamo leggere e scrivere nello stesso ciclo di clock(anche se spesso è irrealistico).

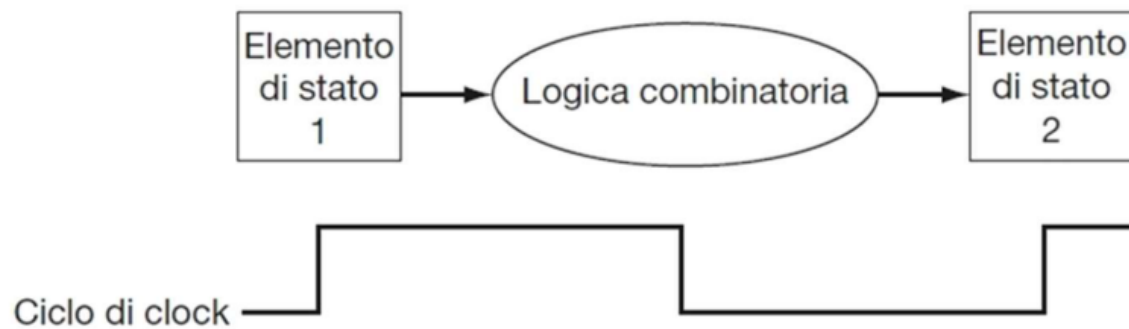


Figure 8: alt text

Il periodo di clock è dato dal tempo massimo che un circuito che compone la CPU ci mette per diventare stabile.

Unità di controllo

L'unità di controllo (in azzurro perchè è un circuito di controllo) comanda tutti i circuiti in base all'istruzione, prende in ingresso l'istruzione da eseguire e dovrà controllare:

- La scrittura per ciascun elemento di stato
- La selezione dei multiplexer
- I segnali di controllo per l'ALU

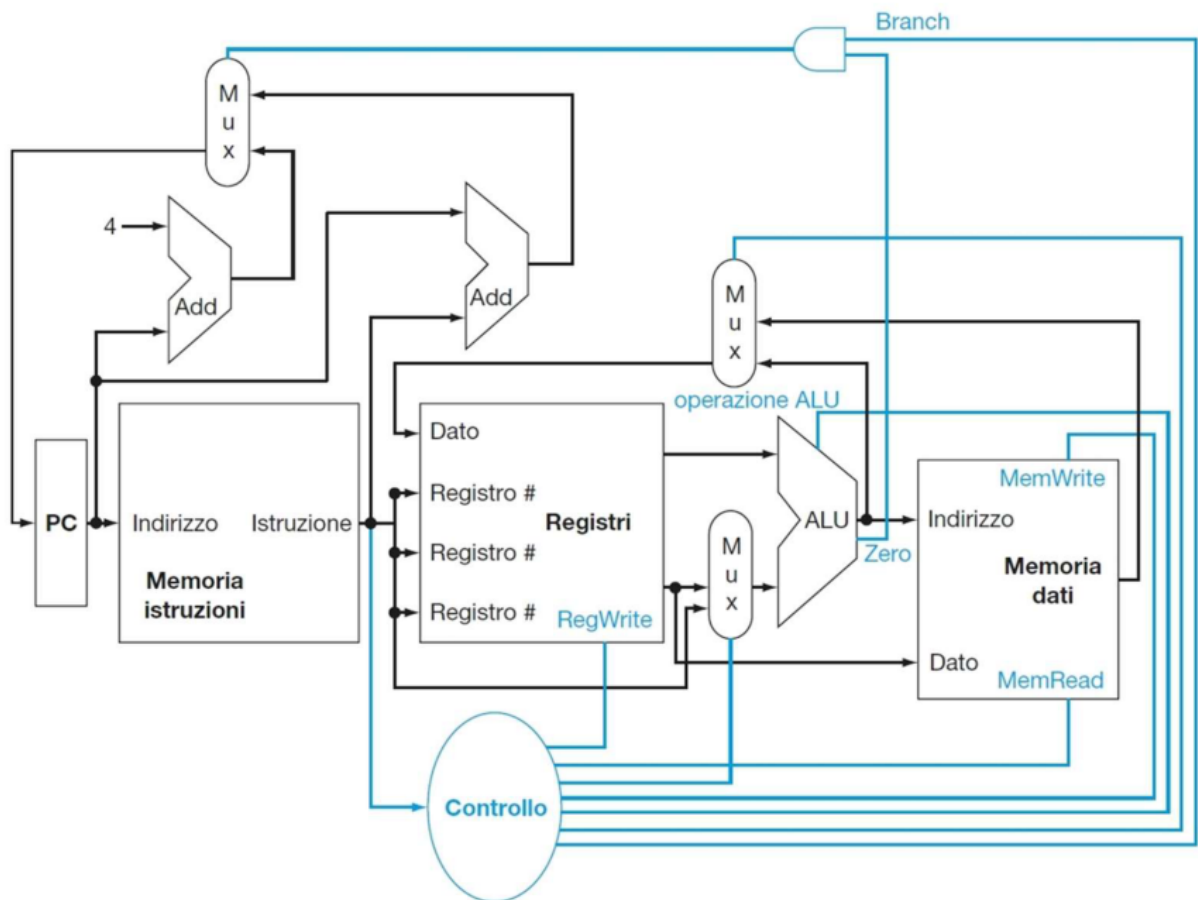


Figure 9: alt text

Ad esempio controlla se scrivere sui registri (nel caso di beq, load, store non è necessario ad esempio.)

Controllo dell'ALU

Nella nostra ALU semplificata abbiamo 4 bit per rappresentare l'ALU operation:

Sequenza	Funzione
0000	AND
0001	OR
0010	ADD

Sequenza	Funzione
0110	subtract
0111	SLT
1100	NOR

Ci occuperemo di fare l'unità di controllo per gestire le prime 4 operazioni.

Per le istruzioni load e store, l'ALU dovrà eseguire la somma di indirizzi e passarla alla memoria dati per la lettura del valore corretto. Per le istruzioni di tipo R(add, sub, and, or) l'ALU dovrà eseguire una di queste 4 operazioni in base al valore dei 7 bit del campo funz7 e dei 3 bit del campo funz3.

Per l'istruzione di salto beq l'alu dovrà fare una sottrazione tra due operandi e controllare se il risultato è 0.

Per comandare l'ALU possiamo scrivere un'unità di controllo che riceve in ingresso i campi funz3 e funz7 e un ingresso da 2 bit chiamato ALUOp che indica se l'operazione da eseguire è una somma(00) o una sottrazione(01), in uscita restituirà 4 bit dell'ALU operation.

L'utilizzo dell'ingresso ausiliario ALUOp che viene controllata dall'unità di controllo principale, viene usata di frequente per ridurre la complessità e la dimensione dell'unità di controllo principale. Inoltre anche la latenza dell'unità di controllo è ridotta in questo modo(infatti spesso rallenta il tempo di clock complessivo per via della latenza).

Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
lw	00	load di 1 parola	XXXXXXX	XXX	somma	0010
sw	00	store di 1 parola	XXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000

Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
Tipo R	10	or	0000000	110	OR	0001

Per progettare questo circuito, scriviamo la tabella di verità per le operazioni RISC-V con il relativo output:

	ALUOp		Campo funz7							Campo funz3			Operazione
	ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
lw / sw	0	0	X	X	X	X	X	X	X	X	X	X	0010
beq	X	1	X	X	X	X	X	X	X	X	X	X	0110
add	1	X	0	0	0	0	0	0	0	0	0	0	0010
sub	1	X	0	1	0	0	0	0	0	0	0	0	0110
and	1	X	0	0	0	0	0	0	0	1	1	1	0000
or	1	X	0	0	0	0	0	0	0	1	1	0	0001

Figure 10: alt text

I termini segnati con una X sono quelli che non influenzano il risultato di questo circuito.

Notiamo che nelle operazioni di add, sub, and e or molti valori del campo funz7 si ripetono (riquadrati in nero), quindi non li consideriamo per compattare il circuito, a questo punto possiamo scrivere le equazioni della tabella di verità in forma normale ed otterremo:

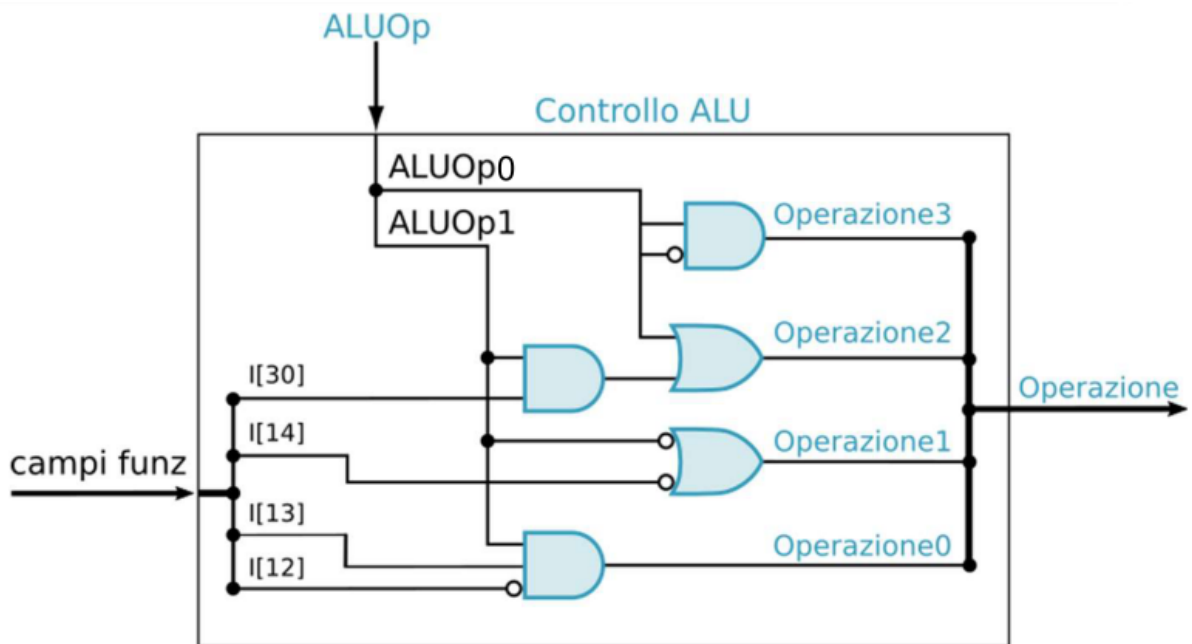


Figure 11: alt text

Unità di controllo principale

Ora che abbiamo progettato l'unità di controllo per l'ALU, procediamo con l'unità di controllo principale, iniziando riguardando i formati delle istruzioni in RISC-V:

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediate[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop

Figure 12: alt text

- Il campo codop è sempre presente, in base a quello l'unità di controllo dovrà stabilire se considerare anche funz3 e funz7 come estensione del codice operativo
- Il primo registro operando(rs1) si trova sempre in posizione 19:15

- Il secondo registro operando(rs2) si trova sempre in posizione 24:20 tranne per le operazioni di tipo I
- Il registro destinazione(rd) si trova sempre in posizione 11:7 per le operazioni I e R

Inoltre:

- Per le operazioni R l'operazione è codificata usando i campi funz3 e funz7
- Per le operazioni I(load), rs1 viene sommato al campo immediato da 12 bit. Il valore presente in memoria a questo indirizzo viene caricato in rd
- Per le operazioni S(store), rs1 viene sommato al campo immediato da 12 bit e da questo indirizzo viene letto un dato, il campo rs2 è il registro sorgente il cui valore viene copiato in memoria.
- Per le istruzioni SB, i registri rs1 e rs2 vanno confrontati, se sono uguali, l'immediato da 12 bit esteso di segno viene shiftato a sinistra di 1 per togliere il segno e sommato/sottratto al PC per calcolare l'indirizzo di destinazione.

Attualmente il nostro processore è così:

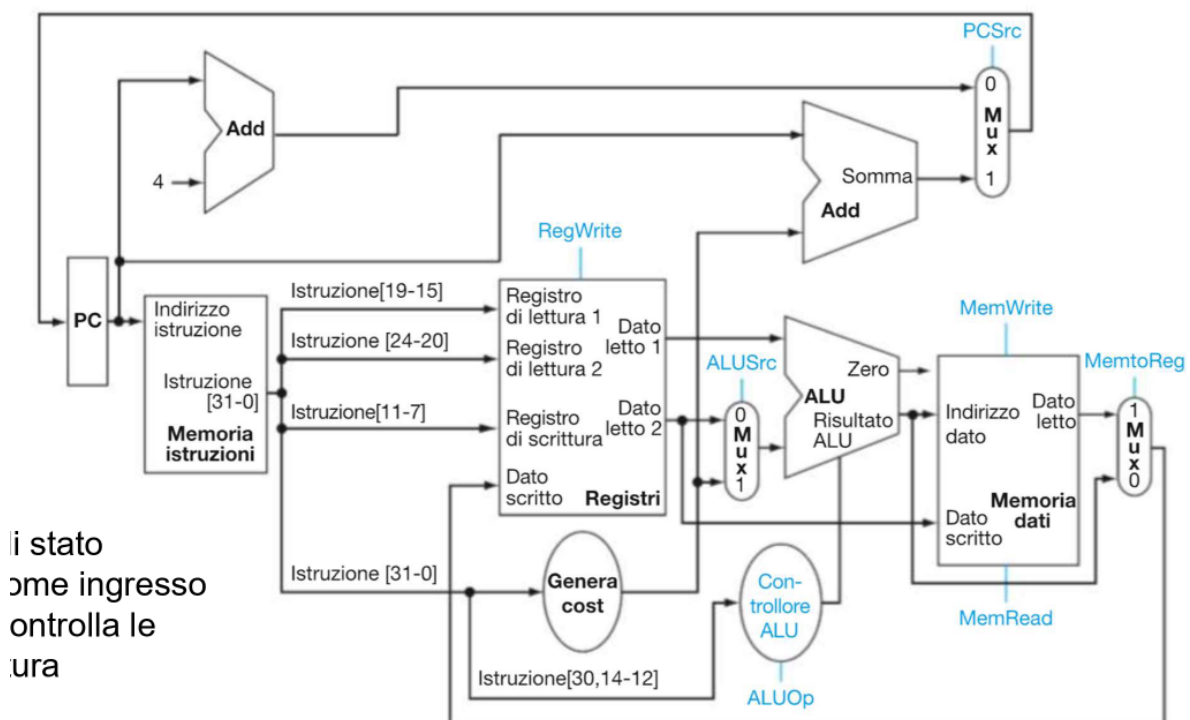


Figure 13: alt text

I segnali di controllo svolgono le seguenti funzioni:

- RegWrite, il dato viene scritto nel registro individuato dal registro di destinazione specificato nell'istruzione
- ALUSrc, se è = 0 il secondo operando dell'ALU è un'immediato proveniente dall'istruzione, altrimenti viene letto dalla seconda uscita del register file
- PCSrc, seleziona l'incremento del PC, nel caso di istruzioni beq dovrà eseguire una somma con immediato, altrimenti semplicemente PC+4
- MemRead, indica se leggere il valore contenuto dalla memoria all'indirizzo dato in input
- MemWrite, indica se scrivere nell'indirizzo dato in input, il valore contenuto in "dato scritto"
- MemToReg, se = 0 il dato inviato al register file per la scrittura proviene dall'ALU, altrimenti proviene dalla memoria.

L'unità di controllo principale gestisce tutti questi ingressi tranne PCSrc basando sul codop.

PCSrc viene gestito sia dall'unità di controllo sia dall'ALU perchè il branch va eseguito se l'istruzione è beq e se la sottrazione tra i due valori è 0, allora mettiamo un AND tra un uscita dell'unità di controllo chiamata branch e l'uscita zero dell'ALU.

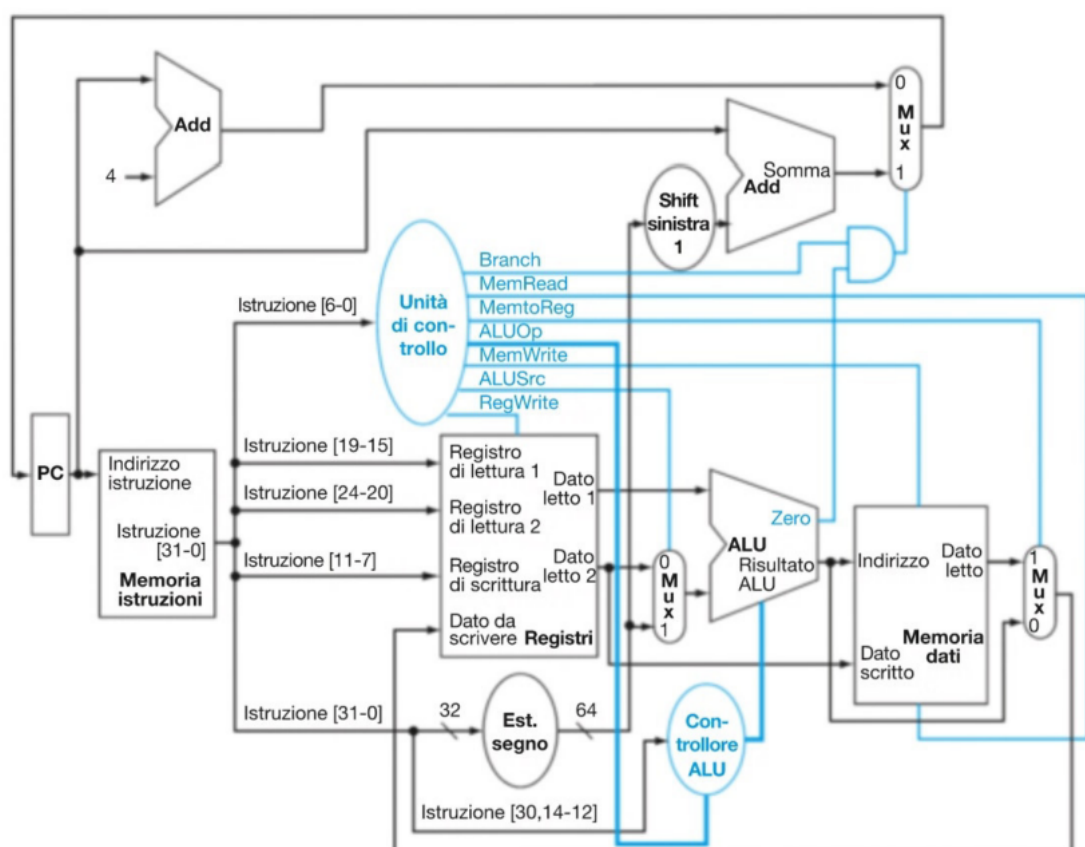


Figure 14: alt text

Una tabella riassuntiva delle linee di controllo in base all'operazione:

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Funzionamento dell'unità di controllo Descriviamo il funzionamento dell'unità di controllo per le varie operazioni:

Istruzioni R

- L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC
- I due registri rs1 e rs2 vengono letti dal register file, l'unità di controllo dovrà impostare le linee di controllo di conseguenza
- L'ALU elabora i dati letti dal register file dato il corretto ALU operation
- Il risultato dell'ALU viene scritto nel registro destinazione usando il register file

Istruzioni Load

- L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC
- Viene letto il contenuto di un registro dal register file
- Viene sommato l'immediato esteso di segno presente nell'istruzione(offset)
- Questo valore rappresenterà l'indirizzo utilizzato per la memoria dati
- Il dato letto dalla memoria dati va scritto in un registro attraverso il register file

Istruzioni beq

- L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC
- Vengono letti i dati da due registri
- L'ALU esegue la sottrazione del valore dei due registri
- Il valore del PC viene sommato al valore immediato esteso shiftato di 1 con il segno presente nell'istruzione(se è una sottrazione il sommatore verrà impostato di conseguenza)
- Il risultato è l'indirizzo di destinazione del salto
- La linea zero dell'ALU determinerà se eseguire il salto eseguendo PC + immediato

Il bus

Il bus interconnette vari componenti tra di loro, questo crea un problema di temporizzazione, infatti se un componente trasmette sul bus, un altro componente non potrebbe trasmettere contemporaneamente. (schema tradizionale, oggi si usano i controller per più velocità, ecc...)

Il bus è suddiviso in 3 categorie:

- Linee di controllo, linee di controllo (0/1 bit) per controllare gli input di controllo dei componenti (MemWrite, ...)
- Linee dati, linee dove passano i dati dalla memoria. Il numero di linee dati (larghezza del data bus), determina il numero di bit trasmessi alla volta.
- Linee indirizzi, linee che controllano gli indirizzi che vengono richiesti alla memoria (e in generale ai componenti)

Esistono due tipi di bus:

- Bus di sistema, interconnette le CPU, I/O e memoria
- Bus interni al chip, interconnette moduli della CPU

Le regole di temporizzazione/controllo dei componenti per l'utilizzo del bus vengono dette protocolli del bus.

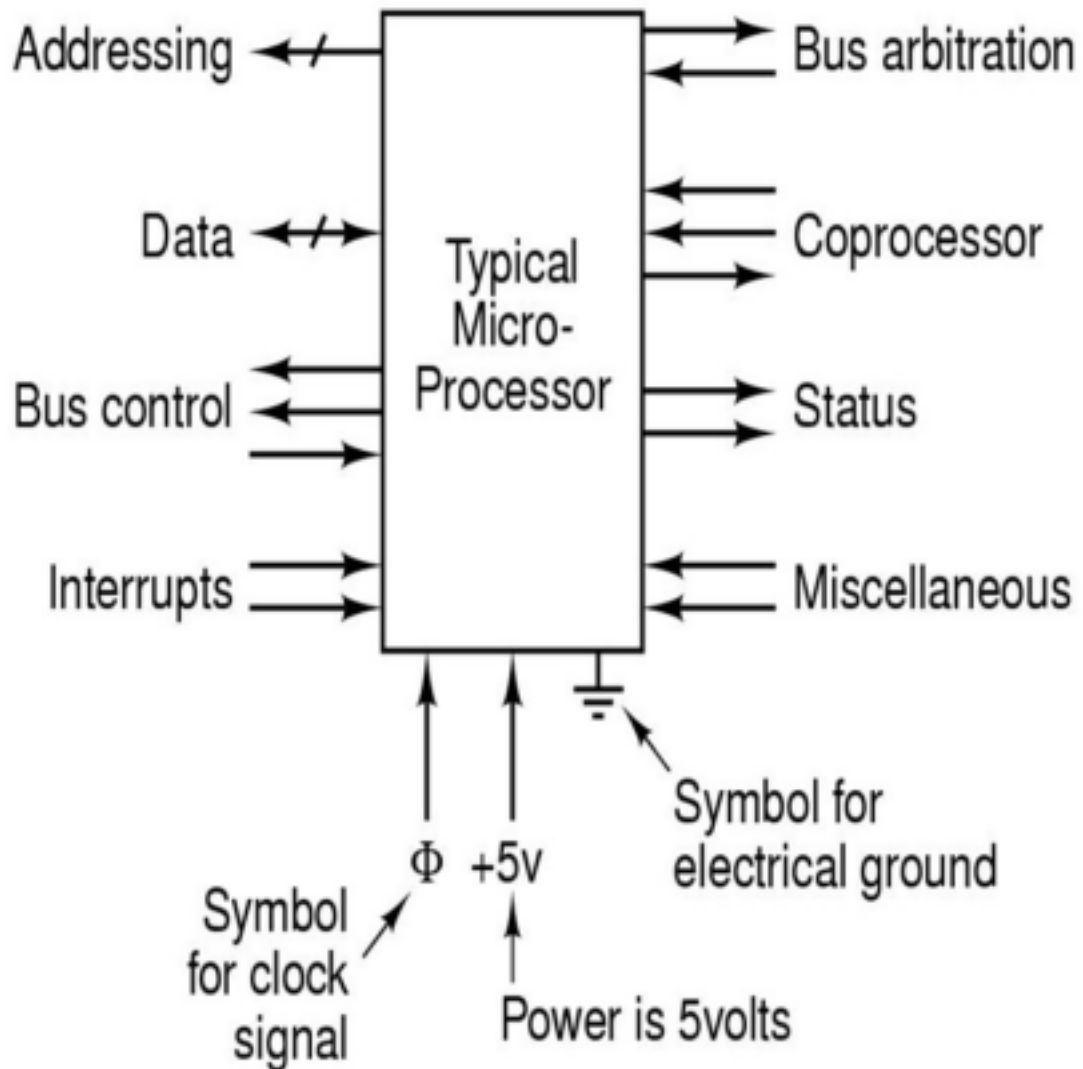


Figure 15: alt text

- Indirizzi, n piedini che corrispondono a 2^n indirizzi che vengono decisi dal processore per richiedere i dati ai componenti
- Dati, n piedini che rappresentano i bit che possono essere letti contemporaneamente
- Controllo, sono dati che il processore manda o riceve(nel caso del I/O) per controllare i vari dispositivi/componenti

Dispositivi attivi e passivi

I dispositivi connessi al bus si dividono in due categorie:

- Bus master, regola il protocollo e i device connessi al bus
- Bus slave, device che vengono regolati master

Master	Slave	Example
CPU	Memoria	Fetching delle istruzioni e dei dati
CPU	Dispositivo I/O	Avvio del trasferimento dei dati
CPU	Coprocessore	CPU che passa un'istruzione al coprocessore
I/O	Memoria	DMA (Direct access memory)
Coprocessore (floating point, ormai integrato nella CPU)	CPU	Coprocessore che recupera gli operandi dalla CPU

Progettazione di un bus

Per progettare un bus bisogna decidere le seguenti caratteristiche:

- Larghezza del bus(numero di linee)
- Arbitraggio, regola l'utilizzo del bus da parte dei vari device per evitare sovrapposizioni
- Funzionamento del bus, come avviene il trasferimento del bus

Bus clocking

Bus sincroni I trasferimenti sul bus sono regolati da un dispositivo di clock, tradizionalmente esterno alla CPU. Ad esempio memorie DDR o dispositivi PCI

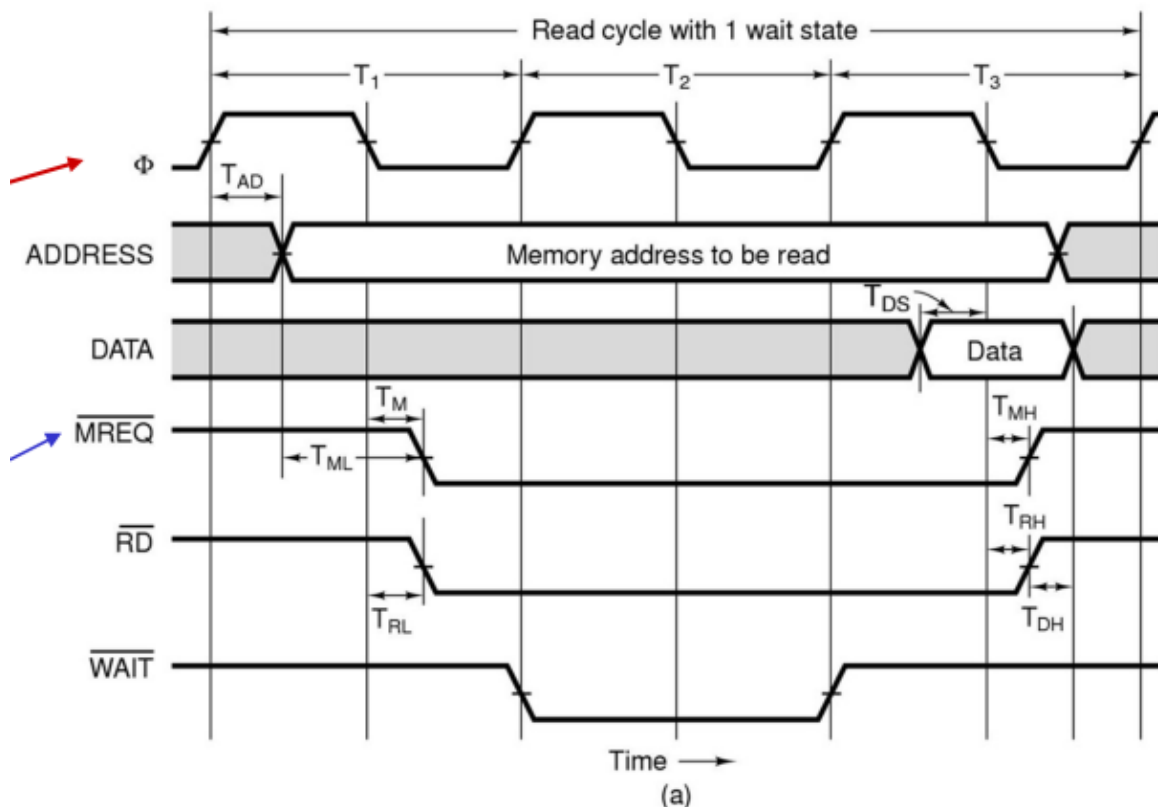


Figure 16: alt text

Questo diagramma temporale, illustra lo scambio di dati tra RAM e CPU.

Abbiamo le seguenti linee:

- Clock
- Address, linee degli indirizzi (è stabile se lo sfondo è bianco (convenzione diagramma))
- Data, linee dei dati
- $\overline{MREQ}$, segnale che indica l'uso della memoria (esclude gli altri componenti connessi al bus)
- $\overline{WAIT}$, segnale che indica al processore quando i dati sul bus dati sono stabili. (0 i dati non sono stabili, 1 i dati possono essere letti)

Notiamo che tutte le operazioni avvengono sul fronte di salita/discesa del clock (talvolta con un po' di delay messo in fase di progettazione).

Bus asincroni I trasferimenti sul bus non sono regolati da un clock ma sono sincronizzati attraverso segnali di handshaking.

Nei sistemi moderni entrambi i tipi di bus sono utilizzati.

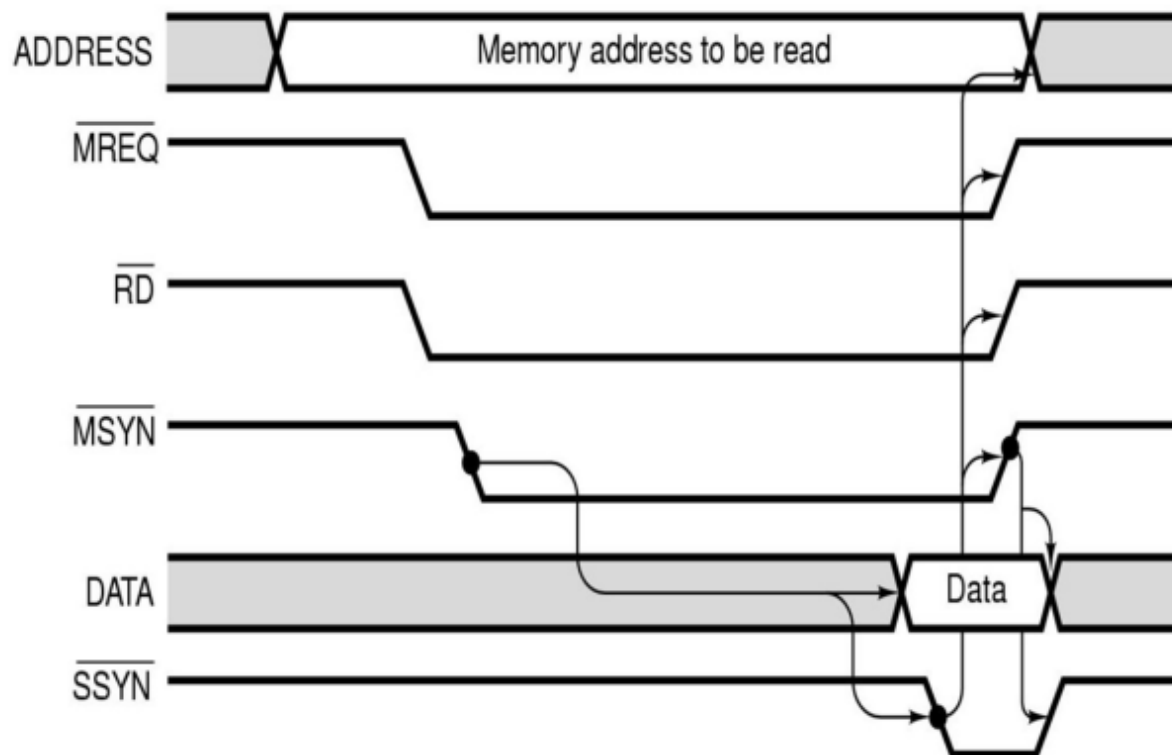


Figure 17: alt text

- Manca la linea di clock
- Vengono usati MSYS(Master synchronization)/SSYN(Slave synchronization), due linee che stabiliscono la sincronia nella comunicazione
- Le altre linee valgono come prima

I segnali MSYS/SSYN vengono messi a valore 1 quando il device sta eseguendo un'operazione e i dati non sono stabili. ### Vantaggi e svantaggi dei tipi di bus

Bus Sincrono

Vantaggi

Svantaggi

Realizzazione device semplice

Vantaggi

Se la durata di un'operazione è fissa non occorre una linea di wait

Svantaggi

Durata di un'operazione di comunicazione deve necessariamente avere una durata pari ad un numero intero di cicli

Bus Asincrono

Vantaggi

Flessibilità: la durata di un'operazione è determinata unicamente dalla velocità della coppia di device

Svantaggi

Per completare un'operazione di comunicazione sono sempre necessarie 4 azioni

Occorre inserire dei device circuiti necessari a rispondere opportunamente al protocollo

Arbitraggio del bus

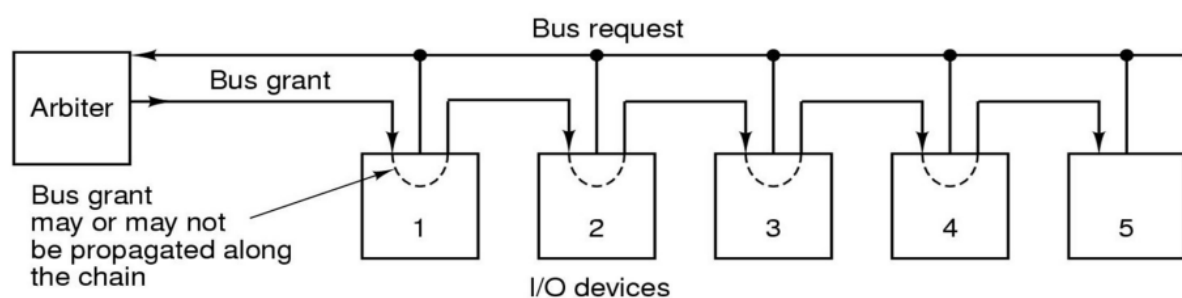


Figure 18: alt text

Arbitraggio centralizzato

- Arbitraggio centralizzato, l'arbitro attiva la grant del bus sulla base della richiesta lungo la linee del grant. Il segnale di attivazione è l'OR del bus grant

Il grant viene trasmesso lungo la linee bus grant finchè un device non accetta l'assegnamento. La priorità viene data al device più vicino, questo dà problemi perchè il device più lontano(nell'esempio 5) potrebbe non avere mai il grant.

Possiamo anche gestire la priorità tra vari device:

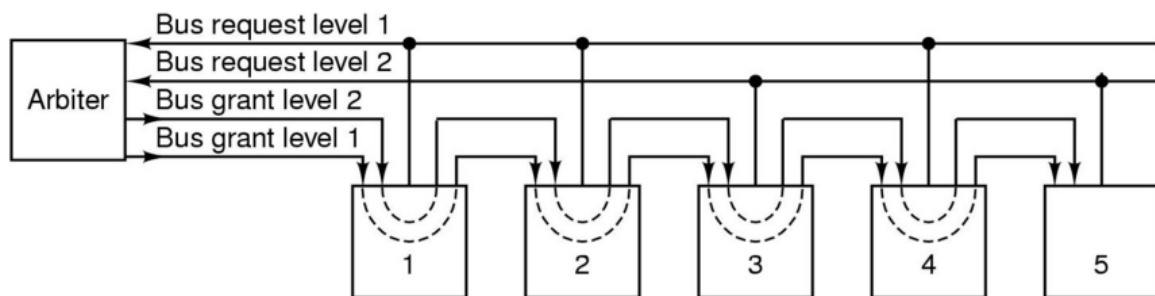


Figure 19: alt text

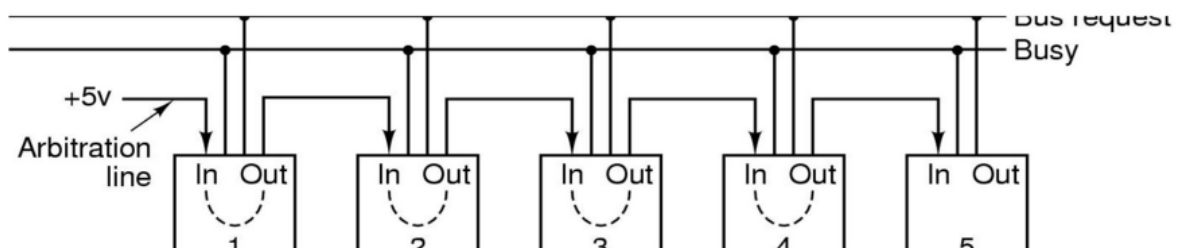


Figure 20: alt text

Arbitraggio decentralizzato Abbiamo due linee:

- Bus request, un componente manda una richiesta che un altro componente deve accettare
- Busy, indica che il bus è in uso
- Arbitration line, attiva il dispositivo

Bus paralleli e seriali

I bus tradizionali(PCI, ISA, ...) utilizzano più linee per trasmettere in parallelo più bit. Tuttavia queste linee hanno un limite fisico(infatti non possono stare tutte vicine per problemi di interferenza), inoltre

non tutti i bit partono allo stesso tempo.

I bus moderni ad alta velocità(USB 3, PCI-E, SATA, ...) trasmettono i dati un bit alla volta ma usando tecniche di codifica e serializzazione raggiungono velocità più alte